Django Tutorial: Writing Your First Django App (Part 1–8)

Introduction

Django is a high-level Python web framework used to build secure, scalable, and maintainable web applications.

In this tutorial, we will build a Polls Application that allows users to view questions, vote on choices, and view results.

Topics Covered

1. Part 1: Creating a Project and App
2. Part 2: Database Setup, Models, ORM, and Admin
3. Part 3: Views and Templates
4. Part 4: Forms and Generic Views
5. Part 5: Automated Testing
6. Part 6: Static Files
7. Part 7: Admin Customization
8. Part 8: Third-party Packages and Django Debug Toolbar

Part 1: Creating a Django Project and App

1. What Is a Django Project?

A Django project is a collection of configuration files and applications that work together to build a website.

Examples:

- Blog website
- E-commerce website
- Polls application
- Learning management system

2. What Is a Django App?

A Django app is a component of a project that performs a specific task.

Example:

```
ecommerce/
├── products/
├── orders/
├── payments/
└── users/
```

A Django project can contain multiple apps.

3. Create a Virtual Environment

```
python -m venv .venv
```

Activate on Windows:

```
.venv\Scripts\activate
```

Activate on Linux/macOS:

```
source .venv/bin/activate
```

4. Install Django

```
python -m pip install Django
```

Check the Django version:

```
python -m django --version
```

5. Create a Django Project

```
django-admin startproject mysite .
```

Project structure:

```
project/
├── manage.py
└── mysite/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Important Files

- `manage.py` : Runs Django management commands.
- `settings.py` : Contains project configuration.
- `urls.py` : Defines URL patterns.
- `asgi.py` : Provides the ASGI application entry point.
- `wsgi.py` : Provides the WSGI application entry point.

6. Run the Development Server

```
python manage.py runserver
```

Open this URL in your browser:

```
http://127.0.0.1:8000/
```

7. Create a Django App

```
python manage.py startapp polls
```

Important files:

- `models.py` : Defines database models.
- `views.py` : Handles requests and responses.
- `admin.py` : Configures Django Admin.
- `tests.py` : Contains automated tests.
- `apps.py` : Contains app configuration.
- `migrations/` : Stores database migration files.

8. Create the First View

`polls/views.py`

```
from django.http import HttpResponse


def index(request):
    return HttpResponse(
        "Hello, world. You're at the polls index."
    )
```

Explanation

- `request` : Contains information about the HTTP request.

- `HttpResponse`: Creates an HTTP response.
- `index()`: A Django view function.

9. Create App URLs

Create `polls/urls.py`.

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name="index"),
]
```

10. Connect App URLs to Project URLs

`mysite/urls.py`

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]
```

Open:

```
http://127.0.0.1:8000/polls/
```

Expected output:

```
Hello, world. You're at the polls index.
```

Part 1 Summary

- Create a Django project.
- Create a Django app.
- Write a view.
- Configure URL routing.
- Run the development server.

Part 2: Database, Models, ORM, and Admin

1. Database Configuration

Django supports databases such as SQLite, PostgreSQL, MySQL, and Oracle.

Example SQLite configuration:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

2. Register the App

In `mysite/settings.py`:

```
INSTALLED_APPS = [
    "polls.apps.PollsConfig",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]
```

Keep any other required apps already present in your project.

3. Apply Existing Migrations

```
python manage.py migrate
```

This command creates or updates database tables according to the migration files.

4. What Is a Model?

A model is a Python class that defines the structure of application data stored in a database.

`polls/models.py`

```
from django.db import models


class Question(models.Model):
```

```
        question_text = models.CharField(max_length=200)
        pub_date = models.DateTimeField("date published")

        def __str__(self):
            return self.question_text


    class Choice(models.Model):
        question = models.ForeignKey(
            Question,
            on_delete=models.CASCADE,
        )
        choice_text = models.CharField(max_length=200)
        votes = models.IntegerField(default=0)

        def __str__(self):
            return self.choice_text
```

Explanation of Model Fields

- `CharField`: Stores text with a specified maximum length.
- `DateTimeField`: Stores date and time.
- `IntegerField`: Stores integer values.
- `ForeignKey`: Defines a relationship between models.
- `on_delete=models.CASCADE`: Deletes related Choice objects when their Question is deleted.
- `default=0`: Sets the initial vote count to zero.
- `__str__()`: Returns a readable string representation of an object.

5. Database Relationship

One Question can have multiple Choices.

```
    Question
    --------
    id
    question_text
    pub_date

           |
           | One-to-Many
           |
```

```
            v

      Choice
      ------
      id
      question_id
      choice_text
      votes
```

6. Create and Apply Migrations

Create migration files:

```
python manage.py makemigrations polls
```

Apply migrations:

```
python manage.py migrate
```

Difference Between the Commands

- `makemigrations`: Creates migration files based on model changes.
- `migrate`: Applies migrations to the database.

7. What Is Django ORM?

ORM stands for Object-Relational Mapping.

Django ORM allows developers to interact with a database using Python instead of writing SQL manually for every operation.

Example:

```
Question.objects.all()
```

Conceptual SQL equivalent:

```
SELECT * FROM polls_question;
```

8. Use the Django Shell

```
python manage.py shell
```

Create a Question:

```
from django.utils import timezone
from polls.models import Question, Choice

q = Question(
    question_text="What's new?",
    pub_date=timezone.now(),
)

q.save()
```

Retrieve its ID:

```
q.id
```

Update the Question:

```
q.question_text = "What's up?"
q.save()
```

9. QuerySet Methods

Get all Questions:

```
Question.objects.all()
```

Filter by ID:

```
Question.objects.filter(id=1)
```

Filter by question text:

```
Question.objects.filter(
    question_text__startswith="What"
)
```

Retrieve one object:

```
Question.objects.get(pk=1)
```

Order by publication date:

```
Question.objects.order_by("-pub_date")
```

Count objects:

```
Question.objects.count()
```

Important QuerySet Methods

- `.all()` : Returns all matching objects.
- `.filter()` : Filters objects.
- `.get()` : Retrieves one object.
- `.order_by()` : Sorts objects.
- `.count()` : Counts objects.
- `.create()` : Creates and saves an object.
- `.delete()` : Deletes objects.

10. Create Choices

```
q.choice_set.create(
    choice_text="Not much",
    votes=0,
)

q.choice_set.create(
    choice_text="The sky",
    votes=0,
)

q.choice_set.create(
    choice_text="Just hacking again",
    votes=0,
)
```

Get all Choices:

```
q.choice_set.all()
```

Count Choices:

```
q.choice_set.count()
```

11. Create a Custom Model Method

```
import datetime

from django.db import models
```

```
    from django.utils import timezone


    class Question(models.Model):
        question_text = models.CharField(max_length=200)
        pub_date = models.DateTimeField("date published")

        def __str__(self):
            return self.question_text

        def was_published_recently(self):
            now = timezone.now()

            return (
                now - datetime.timedelta(days=1)
                <= self.pub_date
                <= now
            )
```

This method returns `True` if the publication time is within the last 24 hours and is not in the future.

12. Create an Admin User

```
python manage.py createsuperuser
```

Register the model in `polls/admin.py`:

```
from django.contrib import admin
from .models import Question

admin.site.register(Question)
```

Open:

```
http://127.0.0.1:8000/admin/
```

Part 2 Summary

- Configure the database.
- Create models.
- Understand model relationships.

- Use migrations.
- Query the database using ORM.
- Manage data through Django Admin.

Part 3: Views and Templates

1. What Is a View?

A view is a Python function or class that receives an HTTP request and returns an HTTP response.

Example:

```
from django.http import HttpResponse


def index(request):
    return HttpResponse("Welcome to Polls")
```

2. URL Parameters

`polls/views.py`

```
from django.http import HttpResponse


def detail(request, question_id):
    return HttpResponse(
        f"You're looking at question {question_id}."
    )


def results(request, question_id):
    return HttpResponse(
        f"You're looking at results of question {question_id}."
    )
```

`polls/urls.py`

```
from django.urls import path
from . import views

app_name = "polls"
```

```
urlpatterns = [
    path("", views.index, name="index"),
    path(
        "<int:question_id>/",
        views.detail,
        name="detail",
    ),
    path(
        "<int:question_id>/results/",
        views.results,
        name="results",
    ),
]
```

The URL pattern `<int:question_id>` captures an integer from the URL.

Example URL:

```
/polls/34/
```

Django calls the view with `question_id=34`.

3. What Is a Template?

A template defines the presentation of a web page, usually using HTML and Django Template Language.

Directory structure:

```
polls/
└── templates/
    └── polls/
        ├── index.html
        └── detail.html
```

4. Create an Index Template

`polls/templates/polls/index.html`

```
{% if latest_question_list %}
    <ul>
        {% for question in latest_question_list %}
            <li>
                <a href="{% url 'polls:detail' question.id %}">
```

```
                        {{ question.question_text }}
                    </a>
                </li>
            {% endfor %}
        </ul>
    {% else %}
        <p>No polls are available.</p>
    {% endif %}
```

Template Syntax

- `{{ variable }}` : Displays a variable.
- `{% if %}` : Performs a conditional check.
- `{% for %}` : Iterates over items.
- `{% url %}` : Generates a URL using its name.
- `{% endif %}` : Ends an if block.
- `{% endfor %}` : Ends a loop.

5. Render a Template

`polls/views.py`

```
from django.shortcuts import render
from .models import Question


def index(request):
    latest_question_list = (
        Question.objects.order_by("-pub_date")[:5]
    )

    context = {
        "latest_question_list": latest_question_list,
    }

    return render(
        request,
        "polls/index.html",
        context,
    )
```

`render()` combines a template with a context and returns an HTTP response.

6. Detail View and 404 Handling

```
from django.shortcuts import get_object_or_404, render
from .models import Question


def detail(request, question_id):
    question = get_object_or_404(
        Question,
        pk=question_id,
    )

    return render(
        request,
        "polls/detail.html",
        {"question": question},
    )
```

`get_object_or_404()` raises an HTTP 404 response if the requested object does not exist.

7. Create a Detail Template

`polls/templates/polls/detail.html`

```
<h1>{{ question.question_text }}</h1>

<ul>
    {% for choice in question.choice_set.all %}
        <li>{{ choice.choice_text }}</li>
    {% endfor %}
</ul>
```

8. URL Namespaces

In `polls/urls.py`:

```
app_name = "polls"
```

Use the URL name in templates:

```
<a href="{% url 'polls:detail' question.id %}">
    {{ question.question_text }}
```

```
    </a>
```

URL names reduce hardcoding and make URL changes easier to manage.

Part 3 Summary

- Understand views and URL parameters.
- Render HTML templates.
- Pass data using context.
- Handle missing objects with 404 responses.
- Use named URLs and namespaces.

Part 4: Forms and Generic Views

1. Create a Voting Form

`polls/templates/polls/detail.html`

```
<form
    action="{% url 'polls:vote' question.id %}"
    method="post"
>
    {% csrf_token %}

    <fieldset>
        <legend>
            <h1>{{ question.question_text }}</h1>
        </legend>

        {% if error_message %}
            <p><strong>{{ error_message }}</strong></p>
        {% endif %}

        {% for choice in question.choice_set.all %}
            <input
                type="radio"
                name="choice"
                id="choice{{ forloop.counter }}"
                value="{{ choice.id }}"
            >

            <label for="choice{{ forloop.counter }}">
```

```
                    {{ choice.choice_text }}
                </label>
                <br>
            {% endfor %}
        </fieldset>

        <input type="submit" value="Vote">
    </form>
```

Important Form Concepts

- `method="post"`: Sends data using a POST request.
- `action`: Defines the destination URL.
- `name="choice"`: Identifies the submitted form field.
- `value`: Contains the selected Choice ID.
- `{% csrf_token %}`: Protects the form against CSRF attacks.

2. Process Votes

`polls/views.py`

```
from django.db.models import F
from django.http import HttpResponseRedirect
from django.shortcuts import get_object_or_404, render
from django.urls import reverse

from .models import Choice, Question


def vote(request, question_id):
    question = get_object_or_404(
        Question,
        pk=question_id,
    )

    try:
        selected_choice = question.choice_set.get(
            pk=request.POST["choice"]
        )
    except (KeyError, Choice.DoesNotExist):
        return render(
            request,
            "polls/detail.html",
```

```
                {
                    "question": question,
                    "error_message": "You didn't select a choice.",
                },
            )
        else:
            selected_choice.votes = F("votes") + 1
            selected_choice.save()

            return HttpResponseRedirect(
                reverse(
                    "polls:results",
                    args=(question.id,),
                )
            )
```

Explanation

- `request.POST["choice"]` : Retrieves the selected Choice ID.
- `try/except` : Handles missing or invalid choices.
- `F("votes") + 1` : Increments the vote count at the database level.
- `reverse()` : Generates a URL using its name.
- `HttpResponseRedirect()` : Redirects the browser to another page.

The `F()` expression helps avoid lost updates when concurrent requests increment the same counter.

3. Results View

```
from django.shortcuts import get_object_or_404, render
from .models import Question


def results(request, question_id):
    question = get_object_or_404(
        Question,
        pk=question_id,
    )

    return render(
        request,
        "polls/results.html",
```

```
            {"question": question},
        )
```

`polls/templates/polls/results.html`

```
<h1>{{ question.question_text }}</h1>

<ul>
    {% for choice in question.choice_set.all %}
        <li>
            {{ choice.choice_text }} --
            {{ choice.votes }}
            vote{{ choice.votes|pluralize }}
        </li>
    {% endfor %}
</ul>

<a href="{% url 'polls:detail' question.id %}">
    Vote again?
</a>
```

4. What Are Generic Views?

Generic views are reusable class-based views that simplify common web development tasks.

Important generic views:

- `ListView`: Displays a list of objects.
- `DetailView`: Displays one object.
- `CreateView`: Creates an object.
- `UpdateView`: Updates an object.
- `DeleteView`: Deletes an object.

5. Use ListView and DetailView

`polls/views.py`

```
from django.views import generic
from .models import Question


class IndexView(generic.ListView):
    template_name = "polls/index.html"
```

```
        context_object_name = "latest_question_list"

        def get_queryset(self):
            return Question.objects.order_by("-pub_date")[:5]


    class DetailView(generic.DetailView):
        model = Question
        template_name = "polls/detail.html"


    class ResultsView(generic.DetailView):
        model = Question
        template_name = "polls/results.html"
```

`polls/urls.py`

```
    from django.urls import path
    from . import views

    app_name = "polls"

    urlpatterns = [
        path("", views.IndexView.as_view(), name="index"),
        path("<int:pk>/", views.DetailView.as_view(), name="detail"),
        path(
            "<int:pk>/results/",
            views.ResultsView.as_view(),
            name="results",
        ),
        path(
            "<int:question_id>/vote/",
            views.vote,
            name="vote",
        ),
    ]
```

`DetailView` normally uses the `pk` URL parameter to identify the object.

Part 4 Summary

- Create HTML forms.
- Process POST requests.

- Protect forms using CSRF tokens.
- Update database values.
- Redirect users after voting.
- Use generic views to reduce boilerplate code.

Part 5: Automated Testing

1. What Is Automated Testing?

Automated testing checks whether application code behaves as expected.

Testing helps detect bugs and prevents existing functionality from breaking when code changes.

2. Test a Model Method

`polls/tests.py`

```
import datetime

from django.test import TestCase
from django.utils import timezone

from .models import Question


class QuestionModelTests(TestCase):
    def test_future_question_is_not_recent(self):
        future_time = timezone.now() + datetime.timedelta(
            days=30
        )

        question = Question(
            question_text="Future question",
            pub_date=future_time,
        )

        self.assertIs(
            question.was_published_recently(),
            False,
        )

    def test_old_question_is_not_recent(self):
```

```
            old_time = timezone.now() - datetime.timedelta(
                days=1,
                seconds=1,
            )

            question = Question(
                question_text="Old question",
                pub_date=old_time,
            )

            self.assertIs(
                question.was_published_recently(),
                False,
            )

        def test_recent_question_is_recent(self):
            recent_time = timezone.now() - datetime.timedelta(
                hours=1
            )

            question = Question(
                question_text="Recent question",
                pub_date=recent_time,
            )

            self.assertIs(
                question.was_published_recently(),
                True,
            )
```

3. Run Tests

```
python manage.py test polls
```

Example successful output:

```
Found 3 test(s).
System check identified no issues.

Ran 3 tests
```

```
OK
```

4. Important Assertions

- `assertEqual(a, b)` : Checks whether two values are equal.
- `assertTrue(x)` : Checks whether a condition is true.
- `assertFalse(x)` : Checks whether a condition is false.
- `assertIs(a, b)` : Checks whether two references point to the same object.
- `assertContains(response, text)` : Checks whether a response contains text.
- `assertEqual(response.status_code, 200)` : Checks the HTTP status code.

5. Test Client

Django's test client simulates HTTP requests without requiring a running external web server.

Example:

```
from django.test import TestCase
from django.urls import reverse


class QuestionIndexViewTests(TestCase):
    def test_no_questions(self):
        response = self.client.get(
            reverse("polls:index")
        )

        self.assertEqual(response.status_code, 200)
        self.assertContains(
            response,
            "No polls are available.",
        )
```

Part 5 Summary

- Write model tests.
- Write view tests.
- Use assertions.

- Simulate requests with the test client.
- Run tests with `manage.py test`.

Part 6: Static Files

1. What Are Static Files?

Static files are files used for the presentation and client-side functionality of a web application.

Examples:

- CSS
- JavaScript
- Images
- Fonts
- Icons

2. Static Directory Structure

```
polls/
└── static/
    └── polls/
        ├── style.css
        └── images/
            └── background.png
```

3. Create a CSS File

`polls/static/polls/style.css`

```
li a {
    color: green;
}
```

4. Load CSS in a Template

`polls/templates/polls/index.html`

```
{% load static %}

<link
    rel="stylesheet"
```

```
        href="{% static 'polls/style.css' %}"
    >
```

- `{% load static %}` loads the static template tag library.
- `{% static %}` generates the URL for a static file.

5. Add a Background Image

```
body {
    background: white url("images/background.png") no-repeat;
}
```

6. Collect Static Files

```
python manage.py collectstatic
```

This command collects static files into the configured destination.

For production, configure an appropriate static file server or storage service.

Part 6 Summary

- Create static directories.
- Add CSS and images.
- Use the static template tag.
- Understand `STATIC_URL` and `collectstatic`.

Part 7: Admin Customization

1. Customize Model Fields

`polls/admin.py`

```
from django.contrib import admin
from .models import Question


class QuestionAdmin(admin.ModelAdmin):
    fields = ["pub_date", "question_text"]


admin.site.register(Question, QuestionAdmin)
```

The `fields` option controls the order in which fields appear in the admin form.

2. Use Fieldsets

```
class QuestionAdmin(admin.ModelAdmin):
    fieldsets = [
        (None, {
            "fields": ["question_text"]
        }),
        ("Date information", {
            "fields": ["pub_date"],
            "classes": ["collapse"],
        }),
    ]
```

Fieldsets organize fields into sections.

3. Add Inline Choices

```
from django.contrib import admin
from .models import Choice, Question


class ChoiceInline(admin.StackedInline):
    model = Choice
    extra = 3


class QuestionAdmin(admin.ModelAdmin):
    fieldsets = [
        (None, {
            "fields": ["question_text"]
        }),
        ("Date information", {
            "fields": ["pub_date"],
            "classes": ["collapse"],
        }),
    ]

    inlines = [ChoiceInline]


admin.site.register(Question, QuestionAdmin)
```

`extra = 3` displays three additional Choice forms.

For a compact table layout:

```
class ChoiceInline(admin.TabularInline):
    model = Choice
    extra = 3
```

Use either `StackedInline` or `TabularInline` in the relevant admin configuration.

4. Customize the List Display

```
class QuestionAdmin(admin.ModelAdmin):
    list_display = [
        "question_text",
        "pub_date",
        "was_published_recently",
    ]
```

5. Add Search and Filters

```
class QuestionAdmin(admin.ModelAdmin):
    list_display = ["question_text", "pub_date"]
    list_filter = ["pub_date"]
    search_fields = ["question_text"]
    date_hierarchy = "pub_date"
```

- `list_display`: Defines columns.
- `list_filter`: Adds filters.
- `search_fields`: Adds a search box.
- `date_hierarchy`: Adds date-based navigation.

6. Customize Admin Site Titles

In `mysite/urls.py`:

```
from django.contrib import admin

admin.site.site_header = "Polls Administration"
admin.site.site_title = "Polls Admin"
admin.site.index_title = "Welcome to Polls Admin"
```

Part 7 Summary

- Customize admin forms.
- Use fieldsets.
- Add inline model forms.
- Configure list display.
- Add search and filters.
- Customize admin titles.

Part 8: Third-party Packages and Django Debug Toolbar

1. What Is a Third-party Package?

A third-party package is a reusable library developed outside the core Python or Django framework.

Examples:

- Django REST Framework
- Django Debug Toolbar
- django-filter
- Pillow
- Celery

2. What Is Django Debug Toolbar?

Django Debug Toolbar is a debugging tool that provides information about the current request and response.

It can help inspect:

- SQL queries
- Query execution time
- Template rendering
- Cache activity
- Request and response information
- Django settings

3. Install Django Debug Toolbar

```
python -m pip install django-debug-toolbar
```

Verify installation:

```
python -m pip show django-debug-toolbar
```

4. Configure Installed Apps

In `mysite/settings.py`:

```
INSTALLED_APPS = [
    # Existing Django apps...
    "debug_toolbar",
]
```

Keep the other apps already present in your project.

5. Configure Middleware

Add the middleware to `MIDDLEWARE` in `settings.py`:

```
MIDDLEWARE = [
    # Existing middleware...
    "debug_toolbar.middleware.DebugToolbarMiddleware",
]
```

Keep the existing middleware and verify the appropriate position against the package's installation documentation.

6. Configure Internal IPs

For local development:

```
INTERNAL_IPS = [
    "127.0.0.1",
]
```

7. Add Debug Toolbar URLs

In `mysite/urls.py`:

```
from django.conf import settings
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]
```

```
if settings.DEBUG:
    import debug_toolbar

    urlpatterns = [
        path("__debug__/", include(debug_toolbar.urls)),
    ] + urlpatterns
```

8. Run the Development Server

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/polls/
```

When correctly configured and enabled for the request, the Debug Toolbar can appear in the browser.

Security note: Never expose Django Debug Toolbar or use `DEBUG=True` in production.

Part 8 Summary

- Install third-party packages.
- Configure apps and middleware.
- Add debugging URLs.
- Inspect SQL queries and request information.
- Use debugging tools only in a safe development environment.

Important Django Commands: Quick Reference

```
# Create virtual environment
python -m venv .venv

# Install Django
python -m pip install Django

# Create project
django-admin startproject mysite .

# Create app
python manage.py startapp polls
```

```
# Run development server
python manage.py runserver

# Check project configuration
python manage.py check

# Create migrations
python manage.py makemigrations

# Apply migrations
python manage.py migrate

# Show migrations
python manage.py showmigrations

# Open Django shell
python manage.py shell

# Create admin user
python manage.py createsuperuser

# Run tests
python manage.py test polls

# Collect static files
python manage.py collectstatic
```

Final Revision: Important Concepts

Concept	Meaning
Project	Main Django configuration and apps

App	A component that performs a specific task
Model	Defines application data
ORM	Interacts with databases using Python
Migration	Records and applies database schema changes
View	Processes requests and returns responses
URLconf	Maps URLs to views
Template	Defines HTML

	presentation
Context	Data passed from a view to a template
Form	Collects user input
CSRF token	Protects supported forms against CSRF attacks
Generic view	Reusable class-based view
TestCase	Django testing class
Static files	CSS, JavaScript, images, and other assets
ModelAdmin	Customizes Django

	Admin
Debug Toolbar	Helps inspect requests, queries, and performance

Conclusion

After completing Parts 1–8, you should understand the basic Django development workflow:

1. Create a project and app.
2. Configure URLs and views.
3. Define models and database relationships.
4. Apply migrations.
5. Retrieve data using the ORM.
6. Render templates.
7. Process forms and POST requests.
8. Write automated tests.
9. Add static files.
10. Customize Django Admin.
11. Use third-party packages for debugging.

Practice Task

Build the Polls application yourself:

1. Create several Questions and Choices through Django Admin.
2. Display the questions on the website.
3. Vote on a question.
4. Display the voting results.
5. Write tests for the main functionality.
6. Use Django Debug Toolbar to inspect database queries.